

括号序列 题解

把 (和) 分别看作 1 和 -1 。

我们称所有的 l_i 和 r_i 为「受限位置」。

$$k = 0$$

此时仅要求合法括号序列，因此答案为 Catalan 数 $C_{n/2}$ 。

直接计算即可，时间复杂度 $\mathcal{O}(n)$ 。

$$n \leq 5000$$

从左往右填。设 $f_{i,j,t}$ 表示考虑到第 i 个位置，当前的前缀和为 j ，上一个 l_i 是 ± 1 的方案数。

直接转移即可，时间复杂度 $\mathcal{O}(n^2)$ 。

$$r_i = l_i + 1$$

此时 $s_{l_i \sim r_i}$ 只能是 () 或)(，并且只与上一个位置的前缀和是否为 0 有关。

于是我们可以在「不是受限位置」并且「前缀和为 0」的位置转移。具体来说，设 f_i 表示考虑到第 i 个位置，当前的前缀和为 0 的方案数。假设从 $j < i$ 转移而来，那么中间的受限位置的方案数为 2 的「 $next_j$ 到 i 中间受限位置的对数」次方，其中 $next_j$ 表示 j 之后的第一个非受限位置，这是因为紧跟着 j 后的受限位置只能是 ()；中间的非受限位置的方案数为 $C_{(i-j)/2-1}$ ，因为我们需要保证中间的前缀和始终 > 0 。当然，这里的 i 和 j 都要是非受限位置。

可以使用分治 NTT 优化，时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

正解

上面的部分提示我们忽略受限位置，按前缀和为 0 的位置转移。我们可以设 f_i 为考虑到第 i 个位置，当前非受限位置的前缀和为 0 的方案数。和上面的做法不同的是，此处因为我们忽略了受限位置对前缀和的贡献，所以前缀和可能为 -1 ，因此我们再设 g_i 表示考虑到第 i 个位置，当前非受限位置的前缀和为 -1 的方案数。

g_i 的转移很简单。首先，只有当 i 和 $i+1$ 都在某一对 (l, r) 中间时，才能允许前缀和 -1 出现，此时 $g_i = f_{pre_i}$ ，其中 pre_i 表示 i 之前的第一个非受限位置，并且此时 (l, r) 这一对只能是 ()。

对于 f_i ，首先可以从 g_{i-1} 转移来，当然也得符合上面的条件；其次也可以像上面的部分分一样，从之前的 f_j 转移过来。但是后者会产生一个问题：我们要计算「 $next_j$ 到 i 中间受限位置的对数」并且把 i 和 j 拆开（因为要分治 NTT）以前缀和作差（ i 之前的 r 数量减去 $next_j$ 之前的 l 数量）的方式计算，所以当 i 和 j 在一对 (l, r) 中间时，会算出来 -1 ，这不是我们想要的。因此，我们在每一步分治 NTT 的时候还需要对受到影响的转移单独进行若干次卷积以修补。

最后直接输出最后一个非受限位置的 f 值即可。